LAB ASSIGNMENT – IX

1. List employee names along with the projects they are working on.

himansh1241 — mysql -u root — 80×16

```
mysql> SELECT e.Name, p.Project_name
    -> FROM Employee e
    -> JOIN Works_On w ON e.Emp_id = w.Emp_id
    -> JOIN Project p ON w.Project_id = p.Project_id;
+------+-----------------+
| Name | Project_name    |
+------+-----------------+
| Amit | AI System       |
| John | AI System       |
| Riya | HR Portal       |
| Sara | Finance Tracker |
| Amit | Cloud Migration |
| John | Cloud Migration |
| Raj  | Cloud Migration |
+------+-----------------+
7 rows in set (0.001 sec)
```

2. Find employees who are not assigned to any project.

himansh1241 — mysql -u root — 80×5

```
mysql> SELECT Name
    -> FROM Employee
    -> WHERE emp_id NOT IN (SELECT emp_id FROM Works_On);
Empty set (0.001 sec)
```

3. Display project names along with total hours worked on each project.

himansh1241 — mysql -u root — 80×13

```
mysql> SELECT p.Project_name, SUM(w.Hours_worked) AS TotalHours
    -> FROM Project p
    -> JOIN Works_On w ON p.project_id = w.project_id
    -> GROUP BY p.Project_name;
+-----------------+------------+
| Project_name    | TotalHours |
+-----------------+------------+
| AI System       |        270 |
| HR Portal       |         80 |
| Finance Tracker |        170 |
| Cloud Migration |        390 |
+-----------------+------------+
4 rows in set (0.001 sec)
```

4. Find employees working on more than one project.

```
himansh1241 — mysql -u root — 80×11
mysql> SELECT Emp_id
    -> FROM Works_On
    -> GROUP BY emp_id
    -> HAVING COUNT(project_id) > 1;
+--------+
| Emp_id |
+--------+
|      1 |
|      3 |
+--------+
2 rows in set (0.001 sec)
```

5. List employees whose department matches the project department they are working on.

```
himansh1241 — mysql -u root — 80×17
mysql> SELECT e.Name, p.Project_name
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> JOIN Project p ON w.project_id = p.project_id
    -> WHERE e.department = p.department;
+------+-----------------+
| Name | Project_name    |
+------+-----------------+
| Amit | AI System       |
| John | AI System       |
| Riya | HR Portal       |
| Sara | Finance Tracker |
| Amit | Cloud Migration |
| John | Cloud Migration |
| Raj  | Cloud Migration |
+------+-----------------+
7 rows in set (0.001 sec)
```

6. Find the average salary of each department.

```
himansh1241 — mysql -u root — 80×11
mysql> SELECT Department, AVG(Salary) AS AvgSalary
    -> FROM Employee
    -> GROUP BY Department;
+------------+------------+
| Department | AvgSalary  |
+------------+------------+
| IT         | 61666.6667 |
| HR         | 50000.0000 |
| Finance    | 65000.0000 |
+------------+------------+
3 rows in set (0.001 sec)
```

7. Find the highest and lowest salary in the company.

```
himansh1241 — mysql -u root — 80×8
mysql> SELECT MAX(Salary) AS Highest, MIN(Salary) AS Lowest
    -> FROM Employee;
+---------+--------+
| Highest | Lowest |
+---------+--------+
|   70000 |  50000 |
+---------+--------+
1 row in set (0.000 sec)
```

8. Calculate total hours worked by each employee.

```
himansh1241 — mysql -u root — 80×14
mysql> SELECT E.Name, SUM(w.Hours_worked) AS TotalHours
    -> FROM Employee e
    -> JOIN Works_On w ON e.emp_id = w.emp_id
    -> GROUP BY e.Name;
+------+------------+
| Name | TotalHours |
+------+------------+
| Amit |        220 |
| Riya |         80 |
| John |        350 |
| Sara |        170 |
| Raj  |         90 |
+------+------------+
5 rows in set (0.001 sec)
```

9. Find the department with the highest average salary.

```
himansh1241 — mysql -u root — 80×11
mysql> SELECT Department
    -> FROM Employee
    -> GROUP BY department
    -> ORDER BY AVG(salary) DESC
    -> LIMIT 1;
+------------+
| Department |
+------------+
| Finance    |
+------------+
1 row in set (0.001 sec)
```

10. Count number of employees in each department.

```
himansh1241 — mysql -u root — 80×11
mysql> SELECT Department, COUNT(*) AS TotalEmployees
    -> FROM Employee
    -> GROUP BY department;
+------------+----------------+
| Department | TotalEmployees |
+------------+----------------+
| IT         |              3 |
| HR         |              1 |
| Finance    |              1 |
+------------+----------------+
3 rows in set (0.001 sec)
```

11. Rank employees based on salary within each department.

```
himansh1241 — mysql -u root — 80×13
mysql> SELECT Name, Department, Salary,
    -> RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS Ranks
    -> FROM Employee;
+------+------------+--------+-------+
| Name | Department | Salary | Ranks |
+------+------------+--------+-------+
| Sara | Finance    |  65000 |     1 |
| Riya | HR         |  50000 |     1 |
| John | IT         |  70000 |     1 |
| Amit | IT         |  60000 |     2 |
| Raj  | IT         |  55000 |     3 |
+------+------------+--------+-------+
5 rows in set (0.001 sec)
```

12. Find the second highest salary using window function.

```
himansh1241 — mysql -u root — 80×12
mysql> SELECT Salary
    -> FROM (
    ->     SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    ->     FROM Employee
    -> ) t
    -> WHERE rnk = 2;
+--------+
| Salary |
+--------+
|  65000 |
+--------+
1 row in set (0.001 sec)
```

13. Calculate running total of salary ordered by join_date.

```
himansh1241 — mysql -u root — 80×13
mysql> SELECT Name, Salary,
    -> SUM(salary) OVER (ORDER BY join_date) AS RunningTotal
    -> FROM Employee;
+------+--------+--------------+
| Name | Salary | RunningTotal |
+------+--------+--------------+
| Sara |  65000 |        65000 |
| John |  70000 |       135000 |
| Amit |  60000 |       195000 |
| Riya |  50000 |       245000 |
| Raj  |  55000 |       300000 |
+------+--------+--------------+
5 rows in set (0.001 sec)
```

14. Find employees whose salary is above department average.

```
himansh1241 — mysql -u root — 80×13
mysql> SELECT Name, Salary
    -> FROM Employee e
    -> WHERE salary > (
    ->     SELECT AVG(salary)
    ->     FROM Employee
    ->     WHERE department = e.department
    -> );
+------+--------+
| Name | Salary |
+------+--------+
| John |  70000 |
+------+--------+
1 row in set (0.001 sec)
```

15. Assign row numbers to employees based on salary descending.

```
himansh1241 — mysql -u root — 80×13
mysql> SELECT Name, Salary,
    -> ROW_NUMBER() OVER (ORDER BY salary DESC) AS RowNum
    -> FROM Employee;
+------+--------+--------+
| Name | Salary | RowNum |
+------+--------+--------+
| John |  70000 |      1 |
| Sara |  65000 |      2 |
| Amit |  60000 |      3 |
| Raj  |  55000 |      4 |
| Riya |  50000 |      5 |
+------+--------+--------+
5 rows in set (0.001 sec)
```

16. Find project with maximum total hours worked.

```
himansh1241 — mysql -u root — 80×11

mysql> SELECT Project_id
    -> FROM Works_On
    -> GROUP BY project_id
    -> ORDER BY SUM(hours_worked) DESC
    -> LIMIT 1;
+------------+
| Project_id |
+------------+
|        104 |
+------------+
1 row in set (0.001 sec)
```

17. Find employees contributing more than average hours in their project.

```
himansh1241 — mysql -u root — 80×14

mysql> SELECT Emp_id, Project_id
    -> FROM Works_On w
    -> WHERE hours_worked > (
    ->     SELECT AVG(hours_worked)
    ->     FROM Works_On
    ->     WHERE project_id = w.project_id
    -> );
+--------+------------+
| Emp_id | Project_id |
+--------+------------+
|      3 |        101 |
|      3 |        104 |
+--------+------------+
2 rows in set (0.001 sec)
```

18. Display employee name, project name, and percentage contribution (hours / total project hours).

```
himansh1241 — mysql -u root — 101×17

mysql> SELECT e.name, p.project_name,
    -> (w.hours_worked * 100.0 / SUM(w.hours_worked) OVER (PARTITION BY w.project_id)) AS percentage
    -> FROM Works_On w
    -> JOIN Employee e ON w.emp_id = e.emp_id
    -> JOIN Project p ON w.project_id = p.project_id;
+------+-----------------+------------+
| name | project_name    | percentage |
+------+-----------------+------------+
| Amit | AI System       |   44.44444 |
| John | AI System       |   55.55556 |
| Riya | HR Portal       |  100.00000 |
| Sara | Finance Tracker |  100.00000 |
| Amit | Cloud Migration |   25.64103 |
| John | Cloud Migration |   51.28205 |
| Raj  | Cloud Migration |   23.07692 |
+------+-----------------+------------+
7 rows in set (0.001 sec)
```

19. Write a procedure to increase salary of employees in a given department by 10%.

himansh1241 — mysql -u root — 87×12

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE IncreaseSalary(IN dept_name VARCHAR(30))
    -> BEGIN
    ->     UPDATE Employee
    ->     SET salary = salary * 1.10
    ->     WHERE department = dept_name;
    -> END //
Query OK, 0 rows affected (0.002 sec)

mysql> DELIMITER ;
mysql> CALL IncreaseSalary('IT');
Query OK, 3 rows affected (0.003 sec)
```

20. Update salary of any employee and verify that the trigger logs the change correctly.

himansh1241 — mysql -u root — 79×16

```
mysql> UPDATE Employee
    -> SET salary = 80000
    -> WHERE emp_id = 1;
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT *FROM Salary_log;
+--------+--------+------------+------------+---------------------+
| Log_id | Emp_id | Old_salary | New_salary | Change_date         |
+--------+--------+------------+------------+---------------------+
|      1 |      1 |      60000 |      66000 | 2026-04-13 11:08:37 |
|      2 |      3 |      70000 |      77000 | 2026-04-13 11:08:37 |
|      3 |      5 |      55000 |      60500 | 2026-04-13 11:08:37 |
|      4 |      1 |      66000 |      80000 | 2026-04-13 11:10:21 |
+--------+--------+------------+------------+---------------------+
4 rows in set (0.000 sec)
```